

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Метрология, стандартизация и сертификация в
информационных технологиях»

ОТЧЕТ

по лабораторной работе №1
«Метрики размера программ»

Подготовили:

Кухто В. С.,
Плехова Е.В.

Проверила:

Болтак С.В.

Минск 2026

Цель работы: Изучить метрики размера программ на примере метрики Холстеда.

Задание: разработать программу на языке программирования Go, содержащую все основные операторы языка, а также пользовательские подпрограммы.

Написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Проанализировать код на заданном языке. На основе анализа программа должна рассчитать метрики Холстеда: 6 базовых и 3 расширенных. Результат работы программы - таблица, как в примере, в которой должны отображаться 6 основных метрик Холстеда. Отдельно выводятся 3 расширенные метрики.

Код анализируемой программы на языке Go:

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)
type operation func(float64, float64) float64
type history struct {
    expr string
    res float64
}
type printer interface {
    print(float64)
}
type consolePrinter struct{}
func (cp consolePrinter) print(val float64) {
    fmt.Printf("Результат: %.2f\n", val)
}
func add(a, b float64) float64 { return a + b }
func sub(a, b float64) float64 { return a - b }
func mul(a, b float64) float64 { return a * b }
func div(a, b float64) float64 {
    if b == 0 {
        fmt.Println("Ошибка: деление на ноль!")
        return 0
    }
    return a / b
}
func isEven(val float64, ch chan string) {
    if int(val)%2 == 0 {
        ch <- "чётное"
```

```

    } else {
        ch <- "нечётное"
    }
}
func main() {
    reader := bufio.NewReader(os.Stdin)
    ops := map[string]operation{
        "+": add,
        "-": sub,
        "*": mul,
        "/": div,
    }
    var hist []history
    var p printer = consolePrinter{}
    defer fmt.Println("\nКалькулятор завершил работу.")
    for {
        fmt.Print("\nВведите выражение (a op b) или 'q' для выхода:
")

        input, _ := reader.ReadString('\n')
        input = strings.TrimSpace(input)
        if input == "q" {
            break
        }
        parts := strings.Fields(input)
        if len(parts) != 3 {
            fmt.Println("Ошибка: введите 3 части (число оператор
число)")
            continue
        }
        a, err1 := strconv.ParseFloat(parts[0], 64)
        b, err2 := strconv.ParseFloat(parts[2], 64)
        if err1 != nil || err2 != nil {
            fmt.Println("Ошибка: неверные числа")
            continue
        }
        op, exists := ops[parts[1]]
        if !exists {
            fmt.Println("Ошибка: неизвестный оператор")
            continue
        }
        ch := make(chan string)
        go isEven(a, ch)
        res := op(a, b)
        p.print(res)
        hist = append(hist, history{expr: input, res: res})
    }
}

```

```

select {
case msg := <-ch:
    fmt.Printf("Число %.2f - %s\n", a, msg)
default:
    fmt.Println("(горутина ещё выполняется...)")
}
switch {
case res < 0:
    fmt.Println("Результат отрицательный")
    fallthrough
case res == 0:
    fmt.Println("Результат равен нулю")
default:
    fmt.Println("Результат положительный")
}
fmt.Println("История (последние 3):")
start := len(hist) - 3
if start < 0 {
    start = 0
}
for i, h := range hist[start:] {
    fmt.Printf(" %d) %s = %.2f\n", start+i+1, h.expr, h.res)
}
}

```

Расчет базовых метрик Холстеда, проведенный вручную, для данной программы приведены в таблице 1.

Таблица 1 – Расчет базовых метрик Холстеда

j	Оператор	f_{1j}	i	Операнд	f_{2i}
1	()	39	1	a	12
2	,	27	2	b	11
3	{}	25	3	res	8
4	float64	15	4	0	8
5	:=	12	5	ch	6
6	:	12	6	input	6
7	fmt.Println	10	7	hist	5
8	.	10	8	parts	5
9	func	8	9	start	5
10	[]	7	10	val	4
11	return	5	11	history	3
12	-	5	12	expr	3
13	<	5	13	consolePrinter	3
14	if	5	14	h	3

15	type	4	15	operation	2
16	string	4	16	printer	2
17	==	4	17	2	2
18	=	4	18	reader	2
19	print	3	19	ops	2
20	fmt.Printf	3	20	p	2
21	+	3	21	strings	2
22	!=	3	22	3	2
23	continue	3	23	err1	2
24	main	2	24	strconv	2
25	struct	2	25	64	2
26	add	2	26	err2	2
27	sub	2	27	nil	2
28	mul	2	28	exists	2
29	div	2	29	1	2
30	if...else	2	30	msg	2
31	isEven	2	31	i	2
32	chan	2	32	"bufio"	1
33	var	2	33	"fmt"	1
34	for...do	2	34	"os"	1
35	len	2	35	"strconv"	1
36	ParseFloat	2	36	"strings"	1
37	op	2	37	cp	1
38	package	1	38	"Результат: %.2f\n"	1
39	import	1	39	"Ошибка: деление на ноль!"	1
40	interface	1	40	"чётное"	1
41	*	1	41	"нечётное"	1
42	/	1	42	bufio	1
43	int	1	43	os	1
44	%	1	44	Stdin	1
45	NewReader	1	45	"+"	1
46	map	1	46	"_"	1
47	defer	1	47	"*"	1
48	fmt.Print	1	48	"/"	1
49	ReadString	1	49	"\nКалькулятор завершил работу."	1
50	TrimSpace	1	50	"\nВведите выражение (a op b) или 'q' для выхода: "	1

51	break	1	51	_ -	1
52	Fields	1	52	n	1
53		1	53	"q"	1
54	!	1	54	"Ошибка: введите 3 части (число оператор число)"	1
55	make	1	55	"Ошибка: неверные числа"	1
56	go	1	56	"Ошибка: неизвестный оператор"	1
57	append	1	57	"Число %.2f - %s\n"	1
58	select	1	58	"(горутина ещё выполняется...)"	1
59	switch-case-default	1	59	"Результат отрицательный"	1
60	range	1	60	(" %d) %s = %.2f\n",	1
61	fallthrough	1	61	"Результат равен нулю"	1
			62	"Результат положительный"	1
			63	"История (последние 3):"	1
$\eta_1 = 61$		$N_1 = 268$	$\eta_2 = 63$		$N_2 = 148$

Словарь программы $\eta = \eta_1 + \eta_2 = 61 + 63 = 124$.

Длина программы $N = N_1 + N_2 = 268 + 148 = 416$.

Объем программы $V = N \cdot \log_2 \eta = 416 \cdot 6,9542 = 2892,946$.

Программа-парсер написана на языке Python с использованием фреймворка Flask.

Предварительную работу программы можно увидеть на рисунке 1.

η_1 (словарь операторов)	η_2 (словарь операндов)	N_1 (общее число операторов)	N_2 (общее число операндов)
61	63	268	148

Рисунок 1 – Предварительная работа программы

Таблицу операторов, получившуюся в результате работы программы, можно увидеть на рисунке 2 (скрин разделен на две страницы, чтобы было лучше видно, это одна таблица).

j	Оператор	f _{ij}
1	()	39
2	,	27
3	{ }	25
4	float64	15
5	:=	12
6	:	12
7	fmt.Println	10
8	.	10
9	func	8
10	[]	7
11	return	5
12	-	5
13	<	5
14	if	5
15	type	4
16	string	4
17	==	4
18	=	4
19	print	3
20	fmt.Printf	3
21	+	3
22	!=	3
23	continue	3
24	main	2
25	struct	2
26	add	2
27	sub	2
28	mul	2
29	div	2
30	if...else	2

31	isEven	2
32	chan	2
33	var	2
34	for...do	2
35	len	2
36	parseFloat	2
37	op	2
38	package	1
39	import	1
40	interface	1
41	*	1
42	/	1
43	int	1
44	%	1
45	NewReader	1
46	map	1
47	defer	1
48	fmt.Print	1
49	ReadString	1
50	TrimSpace	1
51	break	1
52	Fields	1
53		1
54	!	1
55	make	1
56	go	1
57	append	1
58	select	1
59	switch...case...default	1
60	fallthrough	1
61	range	1

Рисунок 2 – Таблица операторов

Таблицу операндов, получившуюся в результате работы программы, можно увидеть на рисунке 3 (скрин разделен на две страницы, чтобы было лучше видно, это одна таблица).

i	Операнд	f _i
1	a	12
2	b	11
3	res	8
4	0	8
5	ch	6
6	input	6
7	hist	5
8	parts	5
9	start	5
10	val	4
11	history	3
12	expr	3
13	consolePrinter	3
14	h	3
15	operation	2
16	printer	2
17	2	2
18	reader	2
19	ops	2
20	p	2
21	strings	2
22	3	2
23	err1	2
24	strconv	2
25	64	2
26	err2	2
27	nil	2
28	exists	2
29	1	2
30	msg	2

31	i	2
32	"bufio"	1
33	"fmt"	1
34	"os"	1
35	"strconv"	1
36	"strings"	1
37	cp	1
38	"Результат: %.2f\n"	1
39	"Ошибка: деление на ноль!"	1
40	"чётное"	1
41	"нечётное"	1
42	bufio	1
43	os	1
44	Stdin	1
45	"+"	1
46	"_"	1
47	"a"	1
48	"/"	1
49	"\nКалькулятор завершил работу."	1
50	"\nВведите выражение (a op b) или 'q' для выхода: "	1
51	—	1
52	n	1
53	"q"	1
54	"Ошибка: введите 3 части (число оператор число)"	1
55	"Ошибка: неверные числа"	1
56	"Ошибка: неизвестный оператор"	1
57	"Число %.2f - %s\n"	1
58	"(горутина ещё выполняется...)"	1
59	"Результат отрицательный"	1
60	"Результат равен нулю"	1
61	"Результат положительный"	1
62	"История (последние 3):"	1
63	" %d) %s = %.2f\n"	1

Рисунок 3 – Таблица операндов

Расширенные метрики, которые выводятся в программе под таблицей, представлены на рисунке 4.

Расширенные метрики		
η (словарь программы) 124 $\eta = \eta_1 + \eta_2 = 61 + 63$	N (длина программы) 416 $N = N_1 + N_2 = 268 + 148$	V (объем программы) 2892.95 $V = N \times \log_2(\eta) = 416 \times \log_2(124)$

Рисунок 4 – Расширенные метрики

Вывод: в ходе выполнения лабораторной работы было разработано консольное приложение-калькулятор на языке программирования Go, реализующее базовые арифметические операции (+, -, *, /) с поддержкой интерактивного ввода, сохранения истории вычислений и многопоточного выполнения с использованием горутин и каналов. Программа демонстрирует использование всех основных конструкций языка и пользовательский подпрограмм.

Для оценки сложности и качества данного программного средства было разработано графическое приложение-парсер на языке Python с использованием фреймворка Flask. С его помощью были рассчитаны базовые и производные метрики Холстеда, которые позволили количественно охарактеризовать программу с точки зрения её объёма, сложности и потенциальной трудоёмкости разработки. Также метрики Холстеда были рассчитаны вручную, для проверки качества работы парсера. Проведенное тестирование показало, что парсер работает корректно, обнаруживает все операторы и операнды и правильно считает по ним метрики Холстеда.